Семейство языков ML

Организация кода и работа с данными

Standard ML OCaml F#

Ошибки, модули, обобщённые типы, коллекции, ввод-вывод

Основные примеры: OCaml 5.4

План второй части

От обработки ошибок к самостоятельной программе

- Ошибки: исключения, option/result, assert и отладка
- Организация: имена, модули, пакеты и интерфейсы
- Обобщённые типы: объявление, применение и ограничения
- Коллекции, затем работа с консолью и файлами

Порядок тем продолжает первую презентацию

Исключения

raise прерывает обычное вычисление

- exception объявляет собственное исключение
- try … with выбирает обработчик по образцу
- Необработанное исключение передаётся выше

```
exception Invalid_score of int

let validate n =
  if n < 0 then raise (Invalid_score n) else n

let () =
  try Printf.printf "%d\n" (validate (-1)) with
  | Invalid_score n ->
      Printf.printf "Invalid score: %d\n" n
```

Тип функции не перечисляет исключения, которые она может поднять

src/slide_03_exceptions.ml

Ошибки как значения

option и result делают исход явным

- option: Some значение или None
- result: Ok результат или Error причина
- match разбирает все предусмотренные варианты

```
let parse_count s =
  match int_of_string_opt s with
  | None -> Error "Not an integer"
  | Some n when n < 0 -> Error "Negative count"
  | Some n -> Ok n

let () =
  match parse_count "12" with
  | Ok n -> Printf.printf "Count: %d\n" n
  | Error message -> prerr_endline message
```

Возврат result сам по себе не перехватывает исключения

src/slide_04_result.ml

Проверки и отладка

assert проверяет внутреннее предположение

- Ложное условие поднимает Assert_failure
- Printexc помогает исследовать исключения
- ocamldebug работает с байткодом, собранным с -g

```
let average a b = (a +. b) /. 2.

let () =
  Printexc.record_backtrace true;
  assert (average 2. 4. = 3.);
  print_endline "Invariant holds"
```

Обычные assert отключаются через -noassert. assert false сохраняется

src/slide_05_debug.ml

Модули и пространства имён

Модуль объединяет связанные определения

- module … struct … end задаёт структуру
- Score.clamp обращается к имени внутри модуля
- let open действует внутри локального выражения

```
module Score = struct
  let maximum = 100
  let clamp x = max 0 (min maximum x)
end

let () =
  let open Score in
  Printf.printf "%d / %d\n" (clamp 120) maximum
```

open сокращает имена. Установка зависимости выполняется отдельно

src/slide_06_modules.ml

Интерфейс и экспорт

Файл .mli задаёт публичный контракт

- counter.ml реализует модуль Counter
- counter.mli перечисляет доступные клиенту имена
- Абстрактный type t скрывает представление

```
type t
val zero : t
val next : t -> t
val to_int : t -> int
```

Клиент использует Counter.next, но не зависит от того, что t реализован как int

src/interfaces/counter.mli

Пакеты и подключение кода

Языковые модули и внешние зависимости решают разные задачи

- OCaml: opam устанавливает пакеты, Dune описывает сборку
- OCaml: open сокращает путь, .mli ограничивает интерфейс
- SML: structure, signature и functor составляют модульную систему
- F#: namespace и module, open, подписи в .fsi

Пакет может содержать несколько модулей и библиотек

Объявление обобщённого типа

Параметр типа описывает семейство типов

- В 'a box символ 'a обозначает параметр типа
- int box и string box конкретизируют параметр
- Box хранит значение выбранного типа

```
type 'a box = Box of 'a

let count : int box = Box 42
let label : string box = Box "ML"

let unbox (Box x) = x

let () =
  Printf.printf "%d %s\n" (unbox count) (unbox label)
```

Применение конструктора типа не является созданием объекта класса

src/slide_09_generic_types.ml

Использование обобщённой функции

Каждое применение выбирает конкретные типы

- first имеет тип 'a * 'b -> 'a
- Компилятор выводит параметры по аргументу
- В F# параметры можно также записывать в <…>

```
let first (x, _) = x
(* first : 'a * 'b -> 'a *)

let n = first (42, "answer")
let s = first ("ML", true)

let () =
  Printf.printf "%d %s\n" n s;
  assert (n = 42 && s = "ML")
```

Функция сохраняет тип выбранного элемента

src/slide_10_generic_functions.ml

Полиморфизм и изменяемые данные

Одна ячейка хранит значения одного типа

- ref None получает слабую переменную типа
- Запись значения фиксирует тип содержимого
- Функция make_cell создаёт независимые ячейки

```
let make_cell () = ref None

let numbers = make_cell ()
let words = make_cell ()

let () =
  numbers := Some 42;
  words := Some "ML";
  assert (!numbers = Some 42);
  assert (!words = Some "ML");
  print_endline "Two independent cells"
```

Value restriction сохраняет безопасность при изменяемом состоянии

src/slide_11_value_restriction.ml

Выбор коллекции

Структура данных определяет удобные операции

- List: неизменяемая цепочка, удобен последовательный обход
- Array: индексный доступ и изменение элементов
- Set и Map: упорядоченные множества и отображения
- Hashtbl: изменяемое отображение с хешированием

SML Basis также содержит неизменяемые Vector. В F# есть List, Array, Set и Map

Списки и преобразования

filter, map и fold описывают обработку списка

- filter оставляет подходящие элементы
- map строит список новых значений
- fold_left накапливает один результат

```
let values = [1; 2; 3; 4]
let evens = List.filter (fun x -> x mod 2 = 0) values
let squares = List.map (fun x -> x * x) evens
let total = List.fold_left ( + ) 0 squares

let () =
  assert (values = [1; 2; 3; 4]);
  assert (squares = [4; 16]);
  Printf.printf "Sum: %d\n" total
```

Исходный список values сохраняется

src/slide_13_lists.ml

Массивы

Размер фиксирован, элементы можно менять

- Литерал массива записывается как [|…|]
- a.(i) читает элемент, a.(i) <- x меняет его
- Индекс должен лежать внутри границ массива

```
let scores = [|10; 20; 30|]
let same = scores

let () =
  scores.(1) <- 99;
  assert (same.(1) = 99);
  Printf.printf "%d, length %d\n"
    scores.(1) (Array.length scores)
```

Два имени одного массива видят одни и те же изменения

src/slide_14_arrays.ml

Множества и карты

Обновление возвращает новую версию коллекции

- Set устраняет повторяющиеся элементы
- Map связывает ключ с одним значением
- Make получает модуль типа и сравнения

```
module Names = Set.Make (String)
module Scores = Map.Make (String)

let names = Names.of_list ["Ada"; "Ada"; "Bob"]
let old = Scores.singleton "Ada" 10
let updated = Scores.add "Ada" 20 old

let () =
  assert (Names.cardinal names = 2);
  assert (Scores.find "Ada" old = 10);
  Printf.printf "%d\n" (Scores.find "Ada" updated)
```

Set.Make и Map.Make являются функторами: они строят модули

src/slide_15_sets_maps.ml

Хеш-таблицы

Hashtbl обновляет существующую таблицу

- replace задаёт актуальное значение по ключу
- find_opt возвращает Some значение или None
- Порядок обхода не следует считать сортировкой

```
let scores = Hashtbl.create 8

let () =
  Hashtbl.replace scores "Ada" 10;
  Hashtbl.replace scores "Ada" 20;
  assert (Hashtbl.length scores = 1);
  match Hashtbl.find_opt scores "Ada" with
  | Some n -> Printf.printf "%d\n" n
  | None -> print_endline "No entry"
```

add допускает несколько привязок одного ключа. Для обновления удобнее replace

src/slide_16_hashtbl.ml

Консольный ввод-вывод

Каналы связывают программу с внешним миром

- stdin читает ввод, stdout выводит результат
- stderr предназначен для сообщений об ошибках
- Формат Printf согласован с типами аргументов

```
let () =
  print_endline "Enter an integer:";
  match In_channel.input_line stdin with
  | None -> prerr_endline "End of input"
  | Some line ->
      match int_of_string_opt line with
      | Some n -> Printf.printf "Value: %d\n" n
      | None -> prerr_endline "Invalid integer"
```

In_channel.input_line возвращает None при конце ввода

src/slide_17_console.ml

Чтение и запись файлов

with_open_bin ограничивает время жизни канала

- Функция получает открытый канал на время работы
- Канал закрывается и при обычном результате, и при исключении
- input_all читает оставшееся содержимое в строку

```
let write path text =
  Out_channel.with_open_bin path (fun channel ->
    output_string channel text)

let read path =
  In_channel.with_open_bin path In_channel.input_all
```

Открытие через Out_channel.with_open_bin перезаписывает существующий файл

src/slide_18_files.ml

Закрытие ресурсов при ошибке

Fun.protect выполняет завершающее действие

- Сначала открываем ресурс, затем запускаем работу
- finally закрывает канал при обоих исходах
- Ошибку чтения обрабатывает вызывающий код

```
let read_first path =
  let channel = open_in path in
  Fun.protect
    ~finally:(fun () -> close_in_noerr channel)
    (fun () -> input_line channel)
```

Для обычного файла предпочтительна готовая with_open_* обёртка

src/slide_19_cleanup.ml

Итоги и источники

Контракты делают поведение программы понятнее

- Тип результата показывает ожидаемые исходы
- Интерфейс модуля отделяет клиента от реализации
- Полиморфизм позволяет переиспользовать алгоритм
- Коллекцию и способ ввода-вывода выбирают под операции

OCaml 5.4: Manual and standard library
ocaml.org/manual/5.4/index.html

Standard ML ’97 and Basis
smlfamily.github.io/

F# Language Reference
learn.microsoft.com/en-us/dotnet/fsharp/language-referenc

opam documentation
opam.ocaml.org/doc/

Dune documentation
dune.readthedocs.io/en/stable/

Следующая часть: конкурентность, парадигмы, выполнение и память